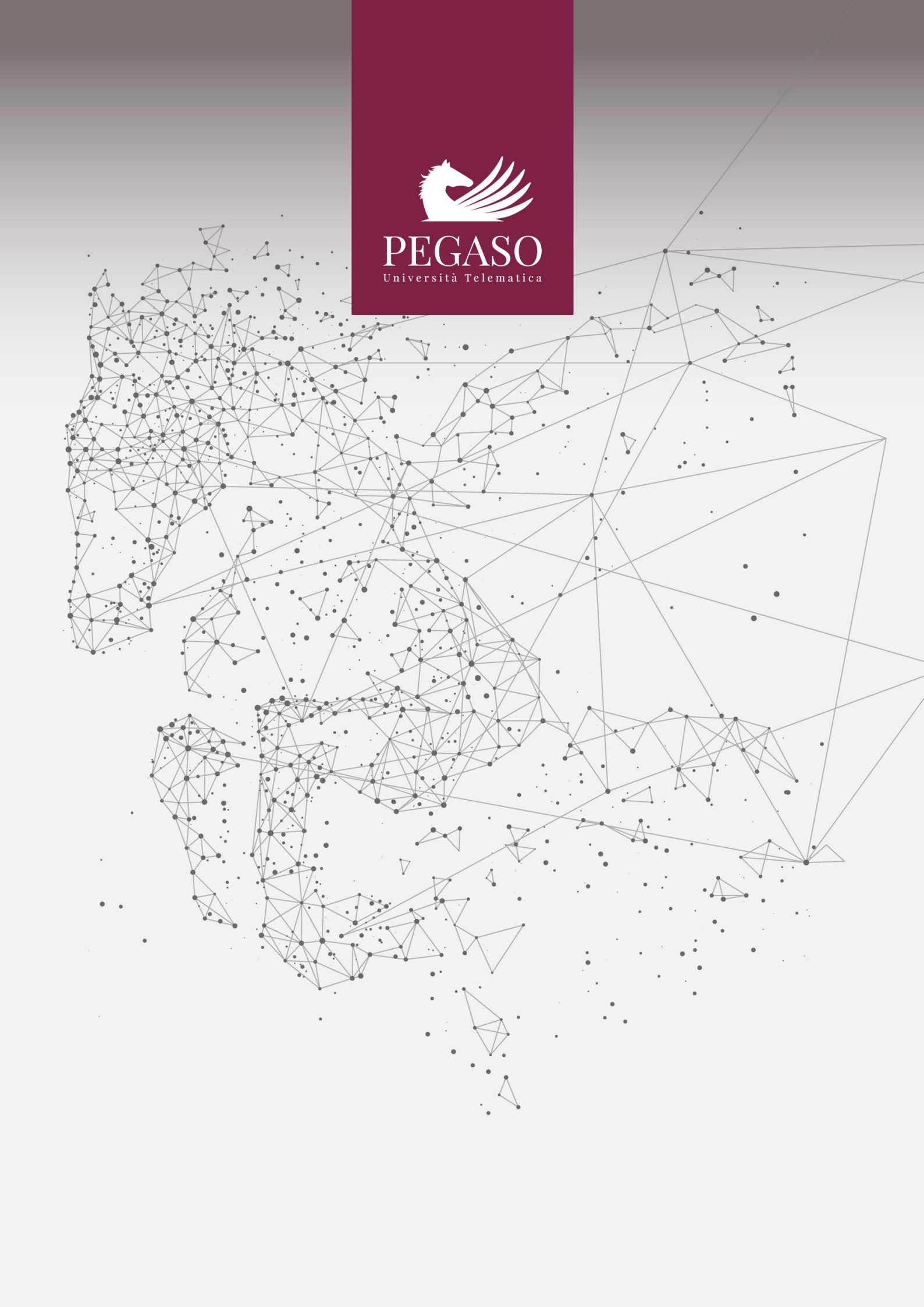

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. VARIABILI E AMBITI DI VISIBILITÀ 3
2. PASSAGGIO DI PARAMETRI CON VARARGS 8
3. COSTRUTTORE 10
4. PACKAGE 14
BIBLIOGRAFIA 16

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Variabili e ambiti di visibilità

Le variabili in Java possono essere dichiarate in diverse posizioni del codice e quindi in diversi contesti:

- dentro una classe
- dentro un metodo
- come parametri di un metodo.

In base al contesto della dichiarazione, si ha un diverso ambito di visibilità ed il ciclo di vita della variabile stessa e si distingue rispettivamente fra:

- variabili di istanza
- variabili locali
- parametri formali

a. Variabili di istanza

Quando le variabili sono definite dentro la dichiarazione di una classe (ma al di fuori di un metodo della classe) esse sono membri della classe (variabile membro per la precisione) e sono dette variabili di istanza o attributi. Hanno in generale valori diversi da istanza ad istanza e il loro valore corrente costituisce lo stato dell'oggetto in un determinato momento. Hanno lo stesso ciclo di vita dell'oggetto istanziato dalla classe in cui è dichiarata e sono inizializzate al momento della creazione dell'istanza dell'oggetto:

- 0 per le variabili numeriche
- false per le variabili di tipo boolean
- null per le variabili di tipo class (oggetti)

Nell'esempio mostrato in Fig. 1 la classe Rettangolo ha come attributi (quindi variabili di istanza) gli interi larghezza, lunghezza e numero_lati (quest'ultima definita come costante). Quando un oggetto della classe Rettangolo è istanziato viene allocato spazio per tutte le sue variabili di istanza che vengono inizializzate ai relativi valori. In questo caso, le variabili altezza e larghezza saranno inizializzate a zero mentre numero_lati è una costante esplicitamente inizializzata a 4 e il suo valore non potrà cambiare.

Una variabile di istanza smetterà di esistere quando smetterà di esistere l'oggetto a cui appartiene.

Si noti che non è possibile comunque definire una variabile al di fuori di una classe.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
public class Rettangolo{
   public int larghezza;
   public int altezza;
   public final int numero_lati = 4;

   public int calcolaArea()
   {
       int area = larghezza * altezza;
       return area;
   }
}
```

Fig. 1: Dichiarazioni delle variabili di istanza larghezza, altezza e numero_lati per la classe rettangolo

b. Variabili locali

Una variabile è detta locale (o di stack o temporanea) se è dichiarata all'interno di un metodo di una classe. Essa smetterà di esistere quanto terminerà il metodo. Una variabile di questo tipo non sarà inizializzata e quindi è buona prassi inizializzare comunque una variabile locale ad un valore di default, nel momento in cui la si dichiara. Infatti il compilatore potrebbe restituire un messaggio di errore laddove ci sia la possibilità che la variabile non venga inizializzata al runtime.

Nell'esempio in Fig. 2 ris è una variabile locale di tipo intero, nasce e muore con l'invocazione e la terminazione del metodo.

```
public int somma (int a, int b) {
      int ris=0;
      ris = a+b;
      return ris;
}
```

Fig. 2: Dichiarazioni delle variabili locali a e b al metodo somma

c. Parametri formali

I metodi possono anche prevedere parametri in ingresso e quindi nella dichiarazione dei parametri formali. Si tratta di variabili che sono dichiarate per poterle passare al metodo e sfruttarle per l'esecuzione o implementazione del metodo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In Fig. 3 è mostrato un esempio di dichiarazione ed invocazione di un metodo che calcola la somma di due numeri.

```
public class Calcolatore
{
   public int somma (int a, int b)
   {
      int ris=0;
      ris = a+b;
      return ris;
   }
}
..
int risultato = oggetto1.somma (5,6);
..
```

Fig. 3: Dichiarazioni dei parametri formali in ingresso ad un metodo

Nella dichiarazione del metodo tra le parentesi tonde sono indicati i parametri attesi (parametri formali) con i rispettivi tipi. Nell'esempio del metodo somma, si tratta di a e b entrambi dichiarati di tipo int.

Nell'invocazione del metodo sono indicati i valori (parametri attuali, o argomenti) che sono passati al metodo per calcolare il risultato.

I parametri formali di un metodo saranno inizializzati al momento della chiamata del metodo con i valori specificati nella invocazione. Infatti, i valori dei parametri attuali vengono copiati (assegnati) nelle variabili usate come parametri formali del metodo. Il passaggio dei parametri avviene quindi per valore.

Nell'esempio del metodo somma, al momento dell'invocazione sono indicati i valori (5 e 6) passati al metodo per calcolare il risultato. Tali valori sono usati per valorizzare e quindi inizializzare i parametri formali: a varrà 5 e b varrà 6.

Dal momento che il passaggio di parametri avviene sempre per valore allora i parametri attuali potrebbe essere anche variabili a cui sono assegnati valori o direttamente o tramite invocazione di altri metodi. Nel primo caso, avremmo anche potuto scrivere:

int x=5, y=6;

int risultato=somma (a,b);

ed ottenere lo stesso risultato.

Un metodo deve sempre essere chiamato con parametri attuali di tipi compatibili con quelli dei parametri formali (e nell'ordine giusto) I nomi dei parametri formali e delle variabili locali di metodi diversi non interferiscono tra loro

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

d. Ambiti di visibilità

Alle variabili di istanza, variabili locali e parametri formali corrispondono diversi ambiti di visibilità legati al contesto in cui sono dichiarati e anche al metodo e processo di memorizzazione.

L'ambito di visibilità dei parametri formali e delle variabili locali è lo stesso, ovvero l'ambito del metodo in cui sono dichiarati. Infatti, i parametri formali di un metodo sono come variabili locali al corpo del metodo. Quindi una variabile locale e un parametro formale hanno lo stesso ambito di visibilità e ciclo di vita ovvero quello del metodo in cui sono dichiarati. Le differenze sussistono solo nella posizione della dichiarazione ma non nel processo di immagazzinamento in memoria. Le variabili locali sono allocate nel record di attivazione relativo alla chiamata del metodo (ovvero nello stack).

Da questo consegue che i nomi dei parametri formali e delle variabili locali di metodi diversi non interferiscono quindi tra loro, perché l'ambito di visibilità è diverso.

Inoltre, condividendo il ciclo di vita con il metodo in cui sono dichiarate, non ha senso (e non è possibile) anteporre alle variabili locali un modificatore d'accesso come public.

L'ambito di visibilità dei parametri formali e delle variabili locali è diverso da quello delle variabili di istanza. Le variabili di istanza condividono il proprio ciclo di vita con l'oggetto (istanza) cui appartengono e utilizzano modalità di memorizzazione completamente differente. Infatti le variabili di istanza sono allocate nell'aria di memoria che descrive l'oggetto (nello heap).

Il fatto che la modalità di allocazione di memoria di una variabile d'istanza si differenzi completamente dalle modalità di allocazione di memoria di una variabile locale (o di un parametro) fa sì che è possibile assegnare in una stessa classe ad una variabile locale (o un parametro) e ad una variabile d'istanza lo stesso identificatore (ovvero nome).

Abbiamo detto che i nomi dei parametri formali e delle variabili di metodi diversi non interferiscono tra loro perché sono in ambiti di visibilità differenti legati al modo e al processo con cui sono memorizzati. Un metodo non "vede" i parametri e le variabili di un altro (nemmeno del main, che è un metodo come tutti gli altri). I nomi dei parametri formali e delle variabili locali di metodi diversi non interferiscono quindi tra loro

Nell'esempio in Fig. 4, si può vedere come il metodo main e il metodo dammiIlPrimo abbiamo dichiarato gli stessi nomi per le variabili locali, ovvero x e y. Il main li definisce come variabili locali a cui da valore iniziale 20 e 10. Il metodo dammiIlPrimo li dichiara come parametri formali e quindi sono anche variabili locali per quel metodo. I valori in quest'ultimo caso sono quelli indicati nella chiamata, ovvero 10 e 20 rispettivamente dalla variabile locale y ed x del metodo main. Anche se il metodo dammiIlPrimo ha come parametri formali x e y non c'è conflitto, anche ambiti di visibilità differenti. Ad x viene dato valore 10

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ed a y valore 20 all'interno del metodo dammiIlPrimo. Per cui il risultato restituito dal metodo è 10 che viene poi stampato a video secondo il comando println nel main.

```
…
public static void main (String [] args){
int y =10 , x = 20;
int ris = dammiIlPrimo (y , x );
System.out.println(ris); // stampa 10
}
…

private static int dammiIlPrimo (int x , int y){
return x;
}
```

Fig. 4: Variabili locali con ambito di visibilità diverso

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Passaggio di parametri con varargs

In Java è possibile dichiarare metodi con una lista di parametri di lunghezza arbitraria (ovvero di lunghezza non definita a priori) e quindi invocare i metodi con una numero variabile di valori purché delle stesso tipo. Tale lista è denominata varargs, nome contratto che sta per «variable arguments».

La sintassi è un po' particolare e fa uso di tre puntini (come i puntini sospensivi), dopo la dichiarazione del tipo. I puntini sospensivi danno l'idea che il parametro corrispondente può essere presente zero o più volte.

Prima dell'introduzione dei varargs occorreva usare gli array per poter passare un numero variabili di parametri. Con i varargs questo può essere evitato e il processo è semplificato. Tuttavia occorre sottolineare che tale parametro resta comunque un array del tipo specificato e quindi come tali vanno trattati.

Attenzione: solo l'ultimo parametro formale di un metodo può avere questa nuova sintassi. Quindi il varargs deve essere l'ultimo argomento e deve essere l'unico.

Sarà trattato come un array di interi: `int[]`

```
public class CalcolatoreVariabile
{
   public int somma (int ... addendi)
   {
        int numparametri = addendi.lenght;
        System.out.println(«numero parametri: " + numparametri);
        int ris = 0;
        for(int i=0; i<numeroparametri; i++) {
            ris=ris+addendi[x];
        }
        return ris;
        }
}

…
CalcolatoreVariabile oggetto1 = new CalcolatoreVariabile();
System.out.println(oggetto1.somma ()); // stampa 0
System.out.println(oggetto1.somma (5,6)); //stampa 11
System.out.println(oggetto1.somma (5,7,2,1,6,8,1)); //stampa 30
…
```

Fig. 5: Metodi con varargs: numero variabile di argomenti

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In Fig. 5, riprendiamo l'esempio della classe Calcolatore con un metodo che calcola la somma in questo caso ridefinita con un numero variabile di parametri e per questa la rinominiamo come CalcolatoreVariabile.

Il numero variabile di parametri di tipo intero è definito con int … addendi. Come detto ciò corrisponde a dire al compilatore Java che il metodo può essere chiamato con zero o n parametri chiamati addendi. Di conseguenza, il parametro addendi è implicitamente dichiarato come un array di tipo int ovvero int[]. Quindi all'interno del metodo, addendi è usato facendo ricorso alla sintassi degli array, addendi[indice].

In caso nessun parametro è indicato nell'invoazione la lunghezza dell'array è 0.

In questo codice abbiamo usato una notazione più compressa con il println dove si può usare direttamente oggetto1.somma() come argomento.

Invece avremmo potuto usare una notazione più compressa per fare il loop sugli array che vedremo in seguito.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Costruttore

In Java esistono metodi speciali che hanno «proprietà». Tra questi c'è da considerare il metodo costruttore che è un metodo speciale che viene invocato ed eseguito una ed una sola volta al momento della creazione di un oggetto (tramite la primitiva new). Il costruttore possiede le seguenti caratteristiche:

- Ha lo stesso nome della classe (è importante fare attenzione a lettere maiuscole e minuscole)
- Non ha tipo di ritorno

Attenzione: il fatto che non abbia tipo di ritorno non significa che il tipo di ritorno è void, ma che non dichiara alcun tipo di ritorno!).

Un costruttore essendo definito senza un tipo di ritorno non può considerarsi un metodo ordinario.

- È chiamato automaticamente (e solamente) ogni volta che è istanziato un oggetto, relativamente a quell'oggetto
- E' presente in ogni classe
- Solitamente un metodo costruttore viene definito allo scopo di inizializzare le variabili di istanza.

```
public class Punto {
   public Punto () //construttore {
      System.out.println («Costruito un punto!»);
   }

   int x;
   int y;
}

…
Punto punto1; //dichiarazione, si crea un riferimento
Punto1 = new Punto(); //istanza, si invoca il costruttore
…
```

Fig. 7: Invocazione del costruttore

Vediamo come avviene l'invocazione del costruttore di un metodo al momento in cui viene istanziato l'oggetto.

Consideriamo come esempio la classe Punto definita come in Fig. 7 con il costruttore Punto (), con lo stesso nome della classe. In questo caso, si esegue una semplice stampa a video nel momento in cui sarà istanziato un oggetto di questa classe. Analizziamo meglio la sintassi che permette di istanziare oggetti. Per esempio:

Punto punto1: //dichiarazione

Punto1 = new Punto(); //istanza

che è equivalente a:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Punto punto1 = new Punto (); //dichiarazione ed istanza

E' la parola chiave new che istanzia formalmente l'oggetto. Perciò basterebbe la sintassi: new Punto(); per istanziare l'oggetto. In questo modo però l'oggetto appena creato non avrebbe un nome (si dirà reference o riferimento) e quindi non sarebbe utilizzabile. Solitamente quindi, quando si istanzia un oggetto, gli si assegna un reference dichiarato precedentemente. In questo caso, con Punto punto1. Appare quindi evidente che l'ultima parte dell'istruzione Punto() non va interpretata come «nomeDellaClasse con parentesi tonde» bensì come «chiamata al metodo costruttore». In corrispondenza dell'istruzione suddetta, un programma produrrebbe il seguente output: Costruito un Punto!

Questo è l'unico modo per chiamare un costruttore.

```
public class Punto {
   public Punto (int a, int b) //costruttore {
      x = a;
      y = b;
   }

   int x;
   int y;
}
…
Punto punto1; //dichiarazione, si crea un riferimento
Punto1 = new Punto(5,6); //istanza, si invoca il costruttore
System.out.println(punto1.x); // stampa 5
System.out.println(punto1.y); // stampa 6
…
Punto punto2 = new Punto(); //errore
```

Fig. 8: Il metodo costruttore per inizializzare le variabili di istanza

Di solito un costruttore è utilizzato per inizializzare le variabili di istanza di un oggetto. E' quindi possibile codificare il seguente costruttore all'interno della classe Punto.

Con questa classe non sarà più possibile istanziare gli oggetti con la solita sintassi: Punto punto1 = new Punto();

Otterremo infatti un errore di compilazione dal momento che staremo cercando di chiamare un costruttore che non esiste (quello senza parametri)! Questo problema è dovuto alla mancanza del costruttore di default in questa dichiarazione, come vedremo fra poco.

La sintassi da usare sarebbe la seguente:

Punto punto1 = new Punto (5,6);

Questa ci permetterebbe anche di inizializzare l'oggetto senza essere costretti ad utilizzare l'operatore .dot.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La creazione dell'oggetto punto2 invece darebbe errore in quanto non è dichiarato un costruttore che non accetta parametri.

Tuttavia è anche possibile fornire più costruttori all'interno della stessa classe, purché la lista dei parametri formali in ingresso differisca. Per esempio sarà fornire entrambi i costruttori che abbiamo visto alla classe Punto (vedi Fig. 9). In questo modo potremo istanziare oggetti dalla classe Punto in due modi, o usando il primo costruttore:

Punto punto1= new Punto ();

Oppure usando il secondo:

Punto punto1 = new Punto (12, 14);

La possibilità di creare più costruttori della stessa classe è una manifestazione del polimorfismo che sia chiama "overload". Questo permette di scrivere più metodi con lo stesso nome (ma con differente lista di parametri). Ci sono altre regole da studiare ma le approfondiremo a tempo debito.

```
public class Punto {
   public Punto () {
      // questo costruttore che non fa niente
   }

   public Punto (int a, int b) //construttore {
      x = a;
      y = b;
   }
   int x;
   int y;
}

…
Punto punto1= new Punto(); //OK
Punto punto1 = new Punto(12, 14); //OK
…
```

Fig. 9: Più costruttori per la stessa classe

Java, come linguaggio ideato da programmatori per programmatori, ha una caratteristica molto importante: spesso vengono inseriti automaticamente e in modo trasparente dal compilatore Java comandi non inseriti dal programmatore. Se infatti provassimo a compilare una classe sprovvista di costruttore, il compilatore ne fornisce uno implicitamente. Il costruttore inserito non contiene comandi che provocano conseguenze visibili al programmatore. Esso è detto «costruttore di default» e non ha parametri. Per questo negli esempi finora mostrati non abbiamo mai istanziato oggetti passando parametri al costruttore.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Se per esempio scrivessimo la classe Punto come in Fig. 10 in alto, al momento della compilazione il compilatore aggiungerebbe ad essa il costruttore di default. Ecco perché negli esempi precedenti sono stati istanziati gli oggetti con la sintassi

Punto punto1= new Punto();

anche senza la presenza esplicita di costruttori. Se non esistesse il costruttore di default avremmo dovuto imparare prima i costruttori e poi gli oggetti.

Questo è anche un esempio per capire la filosofia del linguaggio e di quanto permette una certa facilità di sviluppo. Il costruttore di default inserito implicitamente dal compilatore Java se non inserito uno dal programmatore, ha permesso di parlare di istanze di oggetti senza per forza dover prima spiegare un concetto tanto singolare quanto il costruttore.

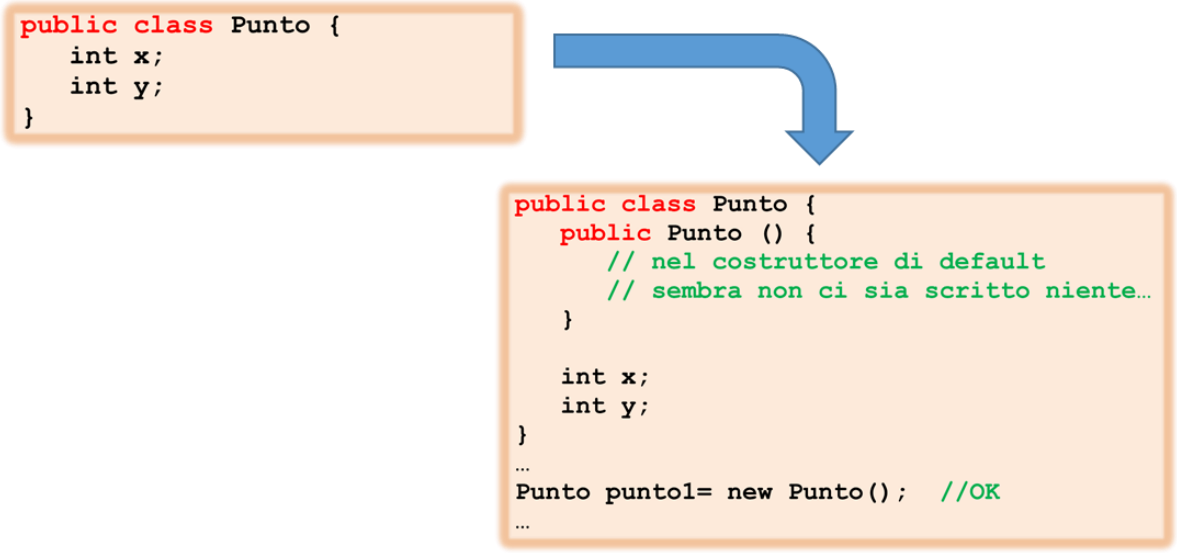


Fig. 10: Costruttore di default

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Package

I package in Java sono un meccanismo che consente di raggruppare le classi. La Libreria Standard di Java (Java API) è organizzata in packages:

- String è una classe del package java.lang
- Scanner è una classe del package java.util

Un package riunisce classi logicamente correlate tra loro, ad esempio:

- Il package java.lang riunisce classi fondamentali del linguaggio Java (String, Math, ...)
- Il package java.util riunisce classi di frequente utilizzo (Scanner, Random, Timer, ...)
- I packages java.awt e java.swing riuniscono classi per costruire interfacce grafiche

I package possono tornare utili quando il programma inizia a diventare complesso e un raggruppamento in packages può consentire di fare ordine e dare un'organizzazione logica alle molte classi.

Un package in Java permette di raggruppare in un'unica entità complessa classi Java logicamente correlate. Fisicamente il package non è altro che una cartella del nostro sistema operativo, ma non tutte le cartelle sono package. Per eleggere una cartella a package, una classe Java deve dichiarare nel suo codice la sua appartenenza a quel determinato package e inoltre deve risiedere fisicamente all'interno di essa. Per esempio se volessimo far appartenere la classe Auto a un package veicoliamotore, dovremmo semplicemente inserire come prima istruzione del nostro file sorgente la seguente dichiarazione di package:

package veicoliamotore;

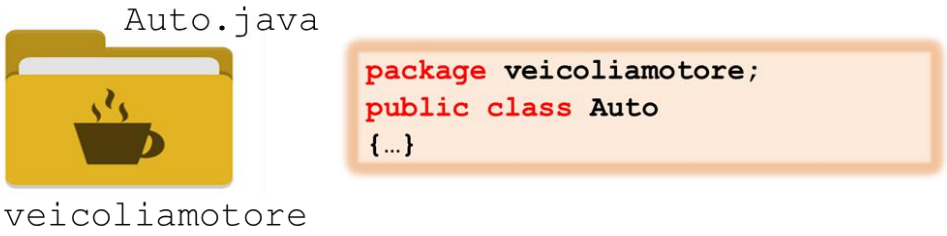


Fig. 11: Uso dei package in Java

Questo comporterebbe che il file compilato .class dovrà risiedere in una cartella chiamata proprio «veicoliamotore».

Se invece volessimo creare un albero di package più strutturato possiamo tranquillamente usare la sintassi sfruttando l'operatore dot (che anche in questo caso significa appartenenza). In sostanza di forma una struttura gerarchica analoga a quella delle directory in un file system.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

package asset.veicoli.veicoliamotore;

In questo caso il file compilato dovrebbe risiedere all'interno di una cartella «veicoliamotore» a sua volta contenuta in una cartella «veicoli» a sua volta contenuta in una cartella «asset».

Il problema con i package sono poi le regole di visibilità delle classi contenute in esse. Come fa una classe che si trova in una cartella a «vedere» una classe che si trova in un'altra cartella? Si usa la direttiva import.

Sicuramente la classe che vuole usare la classe Auto (supponiamo si chiami TestAuto) se non si trova nello stesso package asset.veicoli.veicoli_a_motore deve importare la classe Auto con il comando import che va dichiarato dopo la dichiarazione di package ma prima della dichiarazione della classe , come nell'esempio in Fig.12

```
package asset.veicoli.veicoliamotore;
import asset.veicoli.veicoliammotore.Auto;

public class TestAuto{

public static void main (String args[]) {
Auto auto = new Auto();

…}
}
```

Fig. 12: Visibilità fra classi in diversi package

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- [1] Il nuovo Java, Claudio De Sio Cesari, Hoepli Informatica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).